

I. Prompt Templates

This appendix summarizes the prompt templates used in EMERGE. The templates are grouped by their role in the memory-augmented multi-agent optimization pipeline. Dynamic fields are written in braces, e.g., {problem_description}.

A. Task Classification

1) Prompt Template of MRTA Category Selection: The prompt template in MRTA category selection is shown below.

You are a classifier for MRTA task descriptions.
Directly output one of the following eight classes:

ST_SR_IA, ST_MR_IA, MT_SR_IA, MT_MR_IA,
ST_SR_TA, ST_MR_TA, MT_SR_TA, MT_MR_TA

Classification Rules:

1. ST/MT:
Determine whether a single robot can execute multiple tasks.
If yes, output MT; otherwise, output ST.
2. SR/MR:
Determine whether any task requires multiple robots to collaborate.
If yes, output MR; otherwise, output SR.
3. IA/TA:
Determine whether task planning considers timesteps.
If yes, output TA; otherwise, output IA.

Class Explanation (combined in order):

- First part: ST (Single Task) or MT (Multi-Task)
- Second part: SR (Single Robot) or MR (Multi-Robot)
- Third part: IA (Instantaneous Assignment) or TA (Time-Extended Assignment)

Carefully read the task description, apply the three rules in sequence,
and output only the final class name without any additional text.

Task Description:
{problem_description}

B. Multi-Agent Solver Prompts

1) Prompt Template of Parser: The prompt template for the Parser agent P_{Par} is shown below.

You are an expert in identifying optimization-relevant information
from natural-language problem descriptions.

As the Parser, your task is to analyze the problem description and extract:

- relevant decision variables,
- constraints,
- objectives.

Use your domain expertise to ensure that these elements are accurately
defined and suitable for formulating a solvable LP or MIP model.
Ensure that constraints use only $\leq$, $\geq$, or $=$ operators.
Avoid strict inequalities such as $>$ or $<$.

{example_context}

Review the problem description
{problem_description}

and the comments from other experts
{comments_text}

Then provide the extracted variables, constraints, and objectives
along with their definitions.

When retrieved demonstrations are available, {example_context} is instantiated by one of the following two templates.

The following retrieved example may guide your work:

{example_str}

The following retrieved summary or failure analysis may provide context:

{example_str}

The example string used by this stage follows the format below.

To guide your work, study the following examples carefully.
Each example demonstrates the process from problem description
to structured extraction.

EXAMPLE {i}

Problem Description:
{example_problem}

Example Output:
{example_output}

End of Examples

2) Prompt Template of Modeler: The prompt template for the Modeler agent P_{Mod} is shown below.

You are a modeling expert in Operations Research and Optimization,
specializing in Mixed-Integer Programming (MIP).

Your task is to construct a mathematical optimization model based on:

- the problem description,
- insights provided by other reasoning stages.

Formulate the decision variables, objective, and constraints.
Ensure that the model is comprehensive, solvable, and aligned with
standard operational research principles.

Now the origin problem is as follow:
{problem_description}

And the comments from other reasoning stages are as follow:
{comments_text}

Give your MIP model of this problem.

The model must be a solvable LP or MIP formulation.

All constraints must use only equality, greater-than-or-equal-to, or less-than-or-equal-to relations.

Do not use strict inequalities ($>$ or $<$); replace them with $\backslash\text{geq}$ or $\backslash\text{leq}$ where appropriate.

Your output format should be a JSON like this:

```
{
  "VARIABLES": "A mathematical description of variables",
  "CONSTRAINTS": "A mathematical description of constraints",
  "OBJECTIVE": "A mathematical description of the objective"
}
```

3) Prompt Template of Developer: The prompt template for the Developer agent P_{Dev} is shown below.

You are a Python programmer specializing in operations research and optimization, with expertise in Gurobi.

Your responsibility is to write clean and executable solver code that transforms an optimization formulation into a working Python program. The implementation must align with the problem objective and constraints.

As the Developer, your core task is to translate:

- parsed task information,
- the mathematical formulation,
- the starter code template,

into executable solver code.

All constraints must use only $\leq$, $\geq$, or $=$.

Strict inequalities ($>$ or $<$) are prohibited in LP/MIP formulations.

{example_str}

Analyze the problem step by step based on the provided description
{problem_description}

and the comments from other reasoning stages

{comments_text}.

Using the starter code template

{code_example}

write a Python function that strictly follows the given format and defines a solvable LP or MIP model.

Include only the necessary import statements.

Do not include external test code.

Deliver clean, efficient, and well-structured code.

When retrieved code demonstrations are used, {example_str} is instantiated as follows.

To guide your work, study the following examples carefully.

Each example demonstrates the process from problem description to code implementation.

EXAMPLE {i}

Problem Description:
{example_problem}

Example Code Implementation:
{example_code}

End of Examples

C. Memory Evolution Prompts

1) Prompt Template of Abstract Memory Evolution: The abstraction prompt P_{abs} for updating the abstract memory m_i^* is shown below. It integrates the current episode summary s_{ij} with the previous category-level abstract memory.

You are an experienced work review expert.

You are proficient in summarization, comparison, and iterative writing.

Your core capability is to compare an old summary and a new summary, extract effective patterns, preserve the useful conclusions, and integrate new insights into a more mature and instructive work summary.

You will receive two inputs:

1. Current episode summary:
 {current_summary}
2. Original abstract memory summary:
 {original_summary}

Your Task:

Revise and upgrade the Original Abstract Memory Summary by integrating the new insights and lessons from the Current Episode Summary.

Output Rules:

1. Output only the final revised summary text. No explanations.
2. Preserve reusable category-level modeling knowledge.
3. Integrate useful new patterns, cautions, or corrective lessons.

2) Prompt Template of Successful Episode Summarization: The prompt template for constructing the successful episode summary s_{ij} is shown below.

You are an efficient Model Refinement Expert.

You are skilled at condensing complex optimization models into their core essentials.

Review the following problem and model.

Briefly summarize the problem description, optimization variables, constraints, objective function, and precautions.

Use 1-3 sentences for each item.

Original problem:

{problem_description}

Model file:

{identify_text}

Output Constraints:

1. Each item should be concise.
2. Use general mathematical terminology where appropriate (e.g., binary variables, linear constraints).
3. Do not use mathematical formulas.
4. Do not include redundant explanations.
5. Output the summary directly using the following structure:

[Problem Summary]: [Insert summary here]
 [Optimization Variables]: [Insert summary here]
 [Optimization Constraints]: [Insert summary here]
 [Optimization Objective]: [Insert summary here]
 [Precautions]: [insert precautions here]

3) Prompt Template of Failed Episode Diagnosis: The prompt template for constructing the failed episode diagnosis s_{ij} is shown below.

You are an efficient Model Diagnosis Expert.
 You are skilled at identifying core logical errors in optimization models.

Please analyze the following original problem and a model known to be incorrect.
 Your task is to directly and concisely explain why this modeling approach is wrong.

Original problem:
 {problem_description}

Model file:
 {identify_text}

- Output Requirements:
1. Focus on the root cause:
 Analyze the fundamental logical error or key assumption bias,
 not surface-level or syntax issues.
 2. Use only the following structure.
 3. Do not include an introductory summary.
 4. Explain each part in 1-2 sentences.

Output Structure (Strictly Follow):
 [Error Cause Analysis]
 [Core Error]: [State the most fundamental error in 1-2 sentences]
 [Specific Analysis]: [Explain the contradiction between the model logic and the problem requirements]
 [Potential Consequences]: [State the typical result, e.g., meaningless solution or infeasible model]

D. Baseline Prompt Templates

1) Prompt Template of Direct Standard Prompting: The prompt template in direct standard prompting is shown below.

You are a Python programmer specializing in operations research and optimization.
 Your expertise in using the Gurobi solver is essential for this task.

You are given a specific optimization problem.
 Please develop an efficient Python program that uses Gurobi to address the following problem:

{problem}

Provide the Python code directly.
Ensure that Gurobi is the sole solver used in the solution.

2) Prompt Template of Chain-of-Thought Baseline: The prompt template in chain-of-thought baseline is shown below.

You are a Python programmer specializing in operations research and optimization.
Your expertise in using the Gurobi solver is essential for this task.

You are given a specific problem.
You aim to develop an efficient Python program that addresses the problem.

Now the origin problem is as follow:
{problem_description}

Let's analyse the problem step by step, and then give your Python code.

Here is a starter code:
{code_example}

3) Prompt Template of PHP Baseline: The prompt template in PHP baseline is shown below.

You are a Python programmer in the field of operations research and optimization.
Your expertise in using the Gurobi solver is essential for this task.

You are given a specific problem.
You aim to develop an efficient Python program that addresses the problem.

Now the origin problem is as follow:
{problem_description}

Let's analyse the problem step by step, and then give your Python code.

Here is a starter code:
{code_example}

The code looks like as following:
{history_answer}

You need to check that the code is correct and can be executed directly.